

Cours magistral

Module : Panorama du Cloud et déploiement Azure

Niveau : Mastère Dev Manager Full Stack

Bloc : Optimisation du SI par l'apport du Cloud Computing

Compétences principalement mobilisées : C22, C23, avec liens vers C21, C24 et C25

À retenir

Ce cours prépare directement les apprenants au TP2. Il explique les concepts nécessaires pour comprendre pourquoi et comment décrire une infrastructure Azure sous forme de code avec Terraform.

0 Table des matières

1	Positionnement du cours	4
1.1	Lien avec les compétences du bloc	4
2	Pourquoi l'Infrastructure as Code ?	4
2.1	Le problème des infrastructures créées manuellement	4
2.2	Définition de l'Infrastructure as Code	5
2.3	Bénéfices pour l'entreprise	5
3	Terraform : principes et positionnement	5
3.1	Qu'est-ce que Terraform ?	5
3.2	Déclaratif plutôt qu'impératif	6
3.3	Les providers Terraform	6
4	Le workflow Terraform	6
4.1	Vue d'ensemble	6
4.2	Initialisation : terraform init	7
4.3	Formatage et validation	7
4.4	Prévisualisation : terraform plan	7
4.5	Application : terraform apply	8
4.6	Destruction contrôlée	8
5	Structure d'un projet Terraform	8
5.1	Fichiers principaux	8
5.2	Principe de lisibilité	8
6	Le langage HCL	9
6.1	Blocs, arguments et expressions	9
6.2	Exemple : Resource Group Azure	9
6.3	Références entre ressources	9
7	Variables, locals et outputs	9
7.1	Variables d'entrée	10
7.2	Fichier tfvars	10
7.3	Locals	10
7.4	Outputs	10
8	Le state Terraform	11
8.1	Rôle du state	11
8.2	Risques du state local	11

8.3	State distant sur Azure Storage	11
9	Authentification Terraform vers Azure	11
9.1	Azure CLI en formation	11
9.2	Service principal en contexte professionnel	12
9.3	Gestion des secrets	12
10	Ressources Azure manipulées dans le TP2	12
10.1	Resource Group	12
10.2	Virtual Network et subnets	12
10.3	Network Security Group	13
10.4	Machines virtuelles Linux	13
10.5	Cloud-init et user data	14
10.6	Load Balancer Azure	14
10.7	Storage Account	14
11	Dépendances et graphe Terraform	15
12	Nommage, tags et gouvernance	15
12.1	Pourquoi une convention de nommage ?	15
12.2	Tags obligatoires	15
13	Drift : quand le réel diverge du code	16
13.1	Définition	16
13.2	Exemple de drift	16
13.3	Réduire le drift	16
14	Modules Terraform	16
14.1	Pourquoi modulariser ?	16
14.2	Appeler un module	16
15	Sécurité d'un projet Terraform Azure	17
15.1	Principes clés	17
15.2	Exemples de risques	17
16	FinOps et Terraform	17
16.1	Leviers FinOps applicables au TP2	17
16.2	Exemple d'arbitrage	18
17	Terraform, CI/CD et travail en équipe	18
17.1	Workflow Git recommandé	18
17.2	Rôles dans l'équipe	18

18 Comparaison Terraform, ARM, Bicep et scripts	18
19 Architecture cible du TP2	19
20 Méthode de lecture d'un plan Terraform	19
21 Préparation directe au TP2	20
22 Mini-cas d'application	20
23 Questions de vérification	20
24 Glossaire	21
25 Synthèse pédagogique	21
26 Références officielles utiles	21

1 Positionnement du cours

Objectif pédagogique

À la fin de ce cours, l'apprenant doit être capable d'expliquer les principes de l'Infrastructure as Code, de décrire le fonctionnement de Terraform, d'identifier les ressources Azure manipulées dans le TP2 et de justifier les choix de structuration, de sécurité et de gouvernance d'un projet Terraform.

Le TP2 fait suite au TP1 consacré à la conception d'une architecture cloud Azure. Dans le TP1, l'apprenant raisonne en architecte : il identifie les besoins, choisit les services Azure et produit une architecture cible. Dans le TP2, cette architecture devient reproductible : elle est décrite dans des fichiers de configuration, versionnée, testée, appliquée puis détruite de manière contrôlée.

Le passage de la conception à l'Infrastructure as Code transforme la manière de gérer le SI. L'objectif n'est plus seulement de créer une ressource cloud dans un portail graphique, mais de définir un état attendu, de le partager avec l'équipe, de le relire, de l'auditer et de le rejouer dans plusieurs environnements.

1.1 Lien avec les compétences du bloc

Compétence	Contribution du cours et du TP2
C22	Automatiser la configuration et la gestion des ressources cloud avec Terraform, produire une infrastructure reproductible, versionnée et moins dépendante des manipulations manuelles.
C23	Administrer des ressources cloud au moyen de scripts et de commandes, comprendre la relation entre CLI, fichiers de configuration, state et cloud provider.
C21	Choisir les ressources adaptées à un besoin applicatif et intégrer les contraintes de coût, d'exploitation et d'écoconception.
C24	Préparer la mise en place du monitoring et de l'optimisation à partir d'une infrastructure clairement décrite.
C25	Appliquer des principes de sécurité : moindre privilège, segmentation réseau, contrôle des accès et gestion des secrets.

2 Pourquoi l'Infrastructure as Code ?

2.1 Le problème des infrastructures créées manuellement

Dans un environnement cloud, il est facile de créer des ressources rapidement depuis le portail Azure. Cette facilité est utile pour découvrir un service, mais elle devient risquée lorsqu'une équipe doit gérer des environnements de développement, de test, de préproduction et de production.

Les manipulations manuelles posent plusieurs difficultés :

- elles sont difficiles à rejouer exactement ;
- elles dépendent de la mémoire ou des notes de l'administrateur ;
- elles génèrent des écarts entre environnements ;
- elles rendent les revues de sécurité plus complexes ;

- elles rendent la suppression des ressources plus risquée ;
- elles compliquent l'estimation des coûts.

Point d'attention

Dans un projet réel, deux environnements créés manuellement ne sont presque jamais identiques. Une règle réseau oubliée, un tag absent ou une option de sauvegarde non activée peut créer un incident difficile à diagnostiquer.

2.2 Définition de l'Infrastructure as Code

L'Infrastructure as Code, souvent abrégée IaC, consiste à gérer des ressources d'infrastructure au moyen de fichiers texte. Ces fichiers décrivent l'état attendu des ressources : réseaux, machines virtuelles, bases de données, comptes de stockage, règles de sécurité, identités, politiques et paramètres.

Le code d'infrastructure peut être versionné avec Git, relu dans une pull request, testé dans un environnement temporaire et appliqué de manière répétable. L'infrastructure devient un actif logiciel.

À retenir

L'IaC ne consiste pas seulement à automatiser la création d'une ressource. Elle consiste à rendre l'infrastructure explicite, versionnée, auditable, reproductible et gouvernable.

2.3 Bénéfices pour l'entreprise

Bénéfice	Explication
Reproductibilité	Une même configuration peut être appliquée dans plusieurs environnements.
Traçabilité	Les changements sont suivis dans Git : auteur, date, justification, diff.
Réduction des erreurs	Les actions manuelles répétitives sont remplacées par des fichiers contrôlés.
Standardisation	Les équipes partagent des modèles communs de réseau, sécurité et tags.
Vitesse de livraison	Les environnements peuvent être créés plus rapidement et détruits proprement.
Auditabilité	Les règles de sécurité et de gouvernance sont visibles dans le code.
Optimisation des coûts	Les ressources sont plus faciles à inventorier, taguer, ajuster et supprimer.

3 Terraform : principes et positionnement

3.1 Qu'est-ce que Terraform ?

Terraform est un outil d'Infrastructure as Code qui permet de définir, prévisualiser et appliquer des changements d'infrastructure. Il repose sur des fichiers de configuration écrits en HCL, pour HashiCorp Configuration Language.

Terraform est multi-cloud : le même outil peut piloter Azure, AWS, Google Cloud, Kubernetes, GitHub ou d'autres plateformes, via des plugins appelés providers. Dans le TP2, l'apprenant utilise principalement le provider AzureRM pour gérer des ressources Azure stables telles que Resource Groups, Virtual Networks, subnets, Network Security Groups, interfaces réseau, machines virtuelles et comptes de stockage.

Exemple

Une équipe peut utiliser Terraform pour créer chaque matin un environnement de test, y déployer une application, exécuter une campagne de validation puis supprimer l'environnement en fin de journée. Cette approche limite les coûts et évite les ressources oubliées.

3.2 Déclaratif plutôt qu'impératif

Terraform utilise une approche déclarative. L'utilisateur décrit l'état final souhaité, et Terraform calcule les actions nécessaires pour atteindre cet état. Il ne s'agit pas d'écrire une suite d'ordres du type : créer ceci, puis modifier cela, puis ouvrir ce port. Il s'agit plutôt de dire : l'infrastructure attendue doit contenir ces ressources avec ces propriétés.

Approche	Principe	Exemple
Impérative	Décrit les étapes à exécuter.	Créer un VNet, créer un subnet, attacher un NSG.
Déclarative	Décrit l'état attendu.	Le VNet doit exister avec deux subnets et un NSG associé.

À retenir

Le caractère déclaratif de Terraform explique l'importance du *plan*. Avant d'appliquer, Terraform annonce ce qu'il va créer, modifier ou détruire.

3.3 Les providers Terraform

Un provider est un plugin Terraform qui sait communiquer avec l'API d'une plateforme. Pour Azure, on rencontre principalement :

- **AzureRM** : provider standard pour la majorité des ressources Azure ;
- **AzAPI** : provider plus proche des API Azure Resource Manager, utile pour des fonctionnalités récentes ou spécifiques ;
- **AzureAD** : provider orienté identités Microsoft Entra ID.

Dans un cours d'introduction appliqué au TP2, AzureRM est le choix prioritaire : il est lisible, fortement documenté et adapté aux ressources classiques.

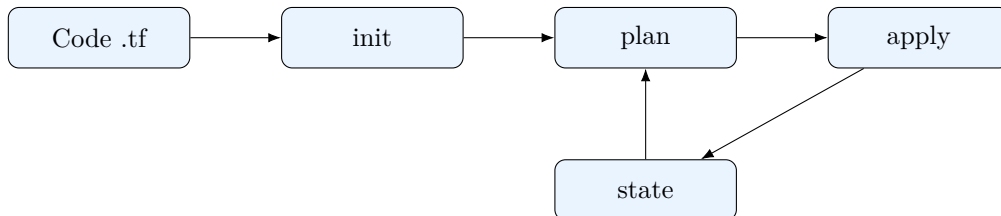
4 Le workflow Terraform

4.1 Vue d'ensemble

Un projet Terraform suit généralement le cycle suivant :

1. écrire ou modifier les fichiers `.tf` ;

2. initialiser le projet avec `terraform init`;
3. formater les fichiers avec `terraform fmt`;
4. vérifier la syntaxe avec `terraform validate`;
5. prévisualiser les actions avec `terraform plan`;
6. appliquer les changements avec `terraform apply`;
7. inspecter les sorties avec `terraform output`;
8. détruire si nécessaire avec `terraform destroy`.



4.2 Initialisation : terraform init

La commande `terraform init` prépare le répertoire de travail. Elle télécharge les providers déclarés, configure le backend si nécessaire et crée un dossier interne `.terraform`. Elle doit être lancée au début du projet ou après une modification significative des providers ou du backend.

```
terraform init
```

4.3 Formatage et validation

Le formatage garantit une présentation homogène des fichiers. La validation détecte les erreurs de syntaxe ou de cohérence avant l'exécution d'un plan.

```
terraform fmt
terraform validate
```

4.4 Prévisualisation : terraform plan

Le plan est une étape centrale. Il compare l'état actuel connu par Terraform avec la configuration souhaitée et indique les changements prévus.

```
terraform plan
```

Les symboles courants du plan sont :

Symbole	Signification
+	Ressource à créer.
-	Ressource à détruire.
~	Ressource à modifier.
-/+	Ressource à remplacer : destruction puis recréation.

Point d'attention

Un plan Terraform doit toujours être lu avant un `apply`. Une destruction imprévue dans le plan est un signal d'alerte.

4.5 Application : terraform apply

La commande `terraform apply` applique les changements. Dans un environnement pédagogique, on peut l'utiliser de manière interactive. Dans une chaîne CI/CD, l'approbation peut être encadrée par un processus de revue.

```
terraform apply
```

4.6 Destruction contrôlée

La commande `terraform destroy` supprime les ressources gérées par le projet. Elle est utile en formation pour éviter les coûts résiduels, mais elle doit être utilisée avec prudence dans un environnement d'entreprise.

```
terraform destroy
```

5 Structure d'un projet Terraform

5.1 Fichiers principaux

Une structure simple et lisible est essentielle pour travailler en équipe. Le TP2 peut utiliser l'organisation suivante :

```
tp2-terraform-azure/  
  main.tf  
  providers.tf  
  variables.tf  
  outputs.tf  
  network.tf  
  compute.tf  
  storage.tf  
  terraform.tfvars.example  
  README.md
```

Fichier	Rôle
providers.tf	Déclare Terraform, les providers et leurs versions.
main.tf	Contient les ressources principales ou l'appel à des modules.
variables.tf	Déclare les paramètres configurables.
outputs.tf	Expose les informations utiles après déploiement.
network.tf	Regroupe les ressources réseau : VNet, subnets, NSG.
compute.tf	Regroupe les ressources de calcul : VM, NIC, IP publiques.
storage.tf	Regroupe les ressources de stockage.
terraform.tfvars.example	Exemple de valeurs, sans secret.
README.md	Documentation d'utilisation du projet.

5.2 Principe de lisibilité

Un projet Terraform doit être compris par une personne qui ne l'a pas écrit. Les noms doivent être explicites, les variables documentées, les tags cohérents et les ressources regroupées par responsabilité.

À retenir

La qualité d'un projet Terraform ne se mesure pas seulement au fait qu'il fonctionne. Elle se mesure aussi à sa lisibilité, sa maintenabilité et sa capacité à être relu sans ambiguïté.

6 Le langage HCL

6.1 Blocs, arguments et expressions

HCL est conçu pour décrire de manière claire des blocs de configuration. Un bloc Terraform suit souvent la forme :

```
resource "type_de_ressource" "nom_logique" {  
  argument = "valeur"  
}
```

Le type de ressource indique l'objet cloud à gérer. Le nom logique est interne au projet Terraform. Il ne correspond pas nécessairement au nom visible dans Azure.

6.2 Exemple : Resource Group Azure

```
resource "azurerm_resource_group" "rg" {  
  name      = "rg-shopeasy-dev"  
  location  = "francecentral"  
  
  tags = {  
    project      = "shopeasy"  
    environment  = "dev"  
    owner        = "formation"  
  }  
}
```

Le Resource Group est une unité logique Azure permettant de regrouper les ressources d'une même solution, d'un même environnement ou d'une même équipe.

6.3 Références entre ressources

Terraform permet de référencer une ressource depuis une autre. Par exemple, une ressource peut utiliser le nom et la localisation du Resource Group :

```
resource "azurerm_virtual_network" "vnet" {  
  name                = "vnet-shopeasy-dev"  
  location             = azurerm_resource_group.rg.location  
  resource_group_name = azurerm_resource_group.rg.name  
  address_space       = ["10.10.0.0/16"]  
}
```

Terraform construit automatiquement un graphe de dépendances. Ici, le VNet dépend du Resource Group, car il en utilise les attributs.

7 Variables, locals et outputs

7.1 Variables d'entrée

Les variables permettent d'éviter les valeurs codées en dur. Elles rendent le projet paramétrable : nom du projet, région, environnement, plage IP, taille de VM, nombre d'instances, etc.

```
variable "environment" {
  description = "Nom de l'environnement"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Region Azure cible"
  type        = string
  default     = "francecentral"
}
```

7.2 Fichier tfvars

Un fichier `terraform.tfvars` peut contenir les valeurs concrètes d'un environnement. Il ne doit jamais contenir de secrets en clair.

```
environment = "dev"
location    = "francecentral"
project     = "shopeasy"
```

7.3 Locals

Les `locals` permettent de calculer des valeurs internes réutilisables, par exemple un préfixe de nommage.

```
locals {
  name_prefix = "${var.project}-${var.environment}"
  common_tags = {
    project      = var.project
    environment  = var.environment
    managed_by   = "terraform"
  }
}
```

7.4 Outputs

Les `outputs` exposent des informations utiles après le déploiement : adresse IP publique, nom du Resource Group, URL de test, identifiant d'un réseau.

```
output "resource_group_name" {
  description = "Nom du Resource Group cree"
  value       = azurerm_resource_group.rg.name
}

output "public_ip" {
  description = "Adresse IP publique de la VM"
  value       = azurerm_public_ip.web.ip_address
}
```

À retenir

Variables : ce qui entre dans le projet. Locals : ce qui est calculé dans le projet. Outputs : ce que le projet expose après déploiement.

8 Le state Terraform

8.1 Rôle du state

Le fichier state est central. Il établit le lien entre les ressources déclarées dans les fichiers Terraform et les ressources réellement créées dans Azure. Sans state, Terraform ne sait pas précisément quelles ressources il gère.

Par défaut, le state est stocké localement dans un fichier `terraform.tfstate`. Cette approche est acceptable en découverte individuelle, mais elle n'est pas adaptée à un travail d'équipe.

8.2 Risques du state local

- perte du fichier state ;
- conflits entre plusieurs administrateurs ;
- impossibilité de verrouiller correctement les changements ;
- présence possible d'informations sensibles ;
- difficulté à intégrer Terraform dans une chaîne CI/CD.

Point d'attention

Le fichier state peut contenir des données sensibles. Il ne doit pas être publié dans un dépôt Git public ou partagé sans contrôle.

8.3 State distant sur Azure Storage

Dans un contexte professionnel, le state est généralement stocké dans un backend distant. Sur Azure, une solution courante consiste à utiliser un Storage Account et un container Blob dédiés au state Terraform.

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "sttfstateprod001"
    container_name      = "tfstate"
    key                 = "shopeasy/dev/terraform.tfstate"
  }
}
```

Le backend distant facilite la collaboration et réduit le risque de perte du state. Il doit être sécurisé par RBAC, chiffrement et restrictions d'accès réseau lorsque le contexte l'exige.

9 Authentification Terraform vers Azure

9.1 Azure CLI en formation

Pour un TP, l'approche la plus simple est souvent l'authentification via Azure CLI :

```
az login
az account show
az account set --subscription "<SUBSCRIPTION_ID>"
```

Terraform peut ensuite utiliser cette session pour appeler les API Azure.

9.2 Service principal en contexte professionnel

En entreprise, Terraform s'exécute souvent depuis une chaîne CI/CD. On utilise alors une identité applicative ou une identité managée, avec des permissions limitées au périmètre nécessaire.

Point d'attention

Attribuer le rôle Owner à un pipeline Terraform est rarement justifié. Une approche plus sûre consiste à limiter les droits au Resource Group ou à l'abonnement cible avec des rôles adaptés.

9.3 Gestion des secrets

Les secrets ne doivent pas être stockés dans les fichiers `.tf`, ni dans un fichier `terraform.tfvars` versionné. Les bonnes pratiques incluent :

- utilisation de variables d'environnement ;
- coffre de secrets tel qu'Azure Key Vault ;
- variables sécurisées dans le système CI/CD ;
- rotation régulière des identifiants ;
- séparation des permissions par environnement.

10 Ressources Azure manipulées dans le TP2

10.1 Resource Group

Le Resource Group regroupe les ressources Azure liées à une application, un environnement ou une équipe. Il sert à organiser, sécuriser, taguer et supprimer collectivement les ressources.

```
resource "azurerm_resource_group" "rg" {
  name      = "rg-${local.name_prefix}"
  location  = var.location
  tags      = local.common_tags
}
```

10.2 Virtual Network et subnets

Le Virtual Network est le réseau privé Azure d'une solution. Les subnets permettent de séparer les composants : couche web, couche applicative, base de données, bastion, services privés.

```
resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-${local.name_prefix}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  address_space       = ["10.20.0.0/16"]
  tags                = local.common_tags
}
```

```
resource "azurerm_subnet" "web" {
  name                = "snet-web"
  resource_group_name = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.20.1.0/24"]
}
```

10.3 Network Security Group

Un Network Security Group, ou NSG, filtre le trafic réseau entrant et sortant au niveau d'une interface réseau ou d'un subnet. Il fonctionne avec des règles de priorité.

```
resource "azurerm_network_security_group" "web" {
  name                = "nsg-web-${local.name_prefix}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  tags               = local.common_tags
}

resource "azurerm_network_security_rule" "allow_http" {
  name                = "allow-http"
  priority            = 100
  direction           = "Inbound"
  access              = "Allow"
  protocol            = "Tcp"
  source_port_range   = "*"
  destination_port_range = "80"
  source_address_prefix = "Internet"
  destination_address_prefix = "*"
  resource_group_name   = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.web.name
}
```

Point d'attention

Ouvrir SSH à Internet avec 0.0.0.0/0 est une mauvaise pratique. En formation, il faut limiter l'accès à l'adresse IP de l'apprenant ou utiliser Azure Bastion si disponible.

10.4 Machines virtuelles Linux

Une machine virtuelle Azure nécessite plusieurs ressources associées : interface réseau, disque, image, taille, identifiants et parfois adresse IP publique.

```
resource "azurerm_linux_virtual_machine" "web" {
  name                = "vm-web-${local.name_prefix}"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  network_interface_ids = [
    azurerm_network_interface.web.id
  ]

  admin_ssh_key {
    username = "azureuser"
  }
}
```

```

    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  tags = local.common_tags
}

```

10.5 Cloud-init et user data

Pour installer automatiquement un serveur web au premier démarrage, on peut utiliser `custom_data` avec un script cloud-init encodé en base64.

```

custom_data = base64encode(<<-INIT
  #cloud-config
  package_update: true
  packages:
    - nginx
  runcmd:
    - systemctl enable nginx
    - systemctl start nginx
    - echo "ShopEasy - VM provisionnée par Terraform" > /var/www/html/index.html
INIT
)

```

10.6 Load Balancer Azure

Un Load Balancer répartit le trafic entre plusieurs machines virtuelles. Il améliore la disponibilité de la couche web lorsque plusieurs instances sont disponibles.

À retenir

Un Load Balancer ne rend pas une application automatiquement hautement disponible. Il doit être combiné avec plusieurs instances, des sondes de santé, une répartition entre zones et une conception applicative adaptée.

10.7 Storage Account

Un Storage Account permet de stocker des données dans Azure : blobs, fichiers, files d'attente ou tables. Dans le TP2, il est utilisé pour illustrer une ressource simple, taguable et paramétrable par Terraform.

```

resource "azurerm_storage_account" "docs" {
  name                = "st${var.project}${var.environment}001"
  resource_group_name = azurerm_resource_group.rg.name
  location             = azurerm_resource_group.rg.location
}

```

```

account_tier          = "Standard"
account_replication_type = "LRS"
min_tls_version       = "TLS1_2"
tags                  = local.common_tags
}

```

11 Dépendances et graphe Terraform

Terraform construit un graphe de dépendances à partir des références entre ressources. Si une machine virtuelle utilise une interface réseau, l'interface doit être créée avant la VM. Si l'interface réseau utilise un subnet, le subnet doit être créé avant l'interface.

La plupart du temps, les dépendances implicites suffisent. On peut toutefois utiliser `depends_on` lorsque la dépendance n'est pas visible dans les attributs.

```

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id          = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}

```

Point d'attention

L'utilisation excessive de `depends_on` peut masquer un mauvais découpage du code. Il faut d'abord chercher à exprimer les dépendances naturellement par les références entre ressources.

12 Nommage, tags et gouvernance

12.1 Pourquoi une convention de nommage ?

Dans Azure, les ressources sont nombreuses. Une convention de nommage permet d'identifier rapidement le projet, l'environnement, le type de ressource et parfois la région.

Exemple

Exemples de noms : `rg-shopeasy-dev`, `vnet-shopeasy-dev`, `snet-web`, `nsg-web-shopeasy-dev`, `vm-web-shopeasy-dev-01`.

12.2 Tags obligatoires

Les tags sont essentiels pour la gouvernance et le FinOps. Ils facilitent les rapports de coûts, l'identification des propriétaires et la gestion du cycle de vie.

Tag	Utilité
<code>project</code>	Identifier l'application ou le produit.
<code>environment</code>	Distinguer dev, test, preprod, prod.
<code>owner</code>	Identifier l'équipe responsable.
<code>cost_center</code>	Affecter les coûts à un budget.
<code>managed_by</code>	Indiquer que la ressource est gérée par Terraform.
<code>expiration</code>	Prévoir la suppression des environnements temporaires.

13 Drift : quand le réel diverge du code

13.1 Définition

Le drift apparaît lorsque l'état réel de l'infrastructure diffère de l'état décrit dans le code Terraform. Cela se produit souvent après une modification manuelle depuis le portail Azure.

13.2 Exemple de drift

Un administrateur ouvre manuellement le port SSH sur un NSG pour dépanner une VM. Le code Terraform ne contient pas cette règle. Au prochain `terraform plan`, Terraform détecte l'écart et propose de revenir à l'état déclaré.

Question de vérification

Pourquoi le drift est-il dangereux dans un SI ? Réponse attendue : parce qu'il rend l'infrastructure moins prévisible, affaiblit la sécurité et réduit la confiance dans le code comme source de vérité.

13.3 Réduire le drift

- interdire ou limiter les modifications manuelles ;
- documenter les procédures d'urgence ;
- exécuter régulièrement `terraform plan` ;
- intégrer Terraform dans les pipelines ;
- utiliser Azure Policy pour encadrer les écarts ;
- former les équipes à modifier l'infrastructure par pull request.

14 Modules Terraform

14.1 Pourquoi modulariser ?

Lorsque le projet grossit, il devient difficile de maintenir de grands fichiers Terraform. Les modules permettent de regrouper une logique réutilisable : module réseau, module VM, module stockage, module monitoring.

```
modules/  
  network/  
    main.tf  
    variables.tf  
    outputs.tf  
  compute/  
    main.tf  
    variables.tf  
    outputs.tf
```

14.2 Appeler un module

```
module "network" {  
  source      = "../modules/network"  
  project     = var.project  
  environment = var.environment
```

```
location      = var.location
address_space = "10.20.0.0/16"
web_subnet_prefix = "10.20.1.0/24"
}
```

À retenir

Un module doit être suffisamment générique pour être réutilisable, mais pas tellement abstrait qu'il devient incompréhensible.

15 Sécurité d'un projet Terraform Azure

15.1 Principes clés

- appliquer le moindre privilège aux identités qui exécutent Terraform ;
- protéger le backend de state ;
- éviter les secrets dans le code ;
- limiter les ports ouverts dans les NSG ;
- activer les logs et diagnostics quand le contexte l'exige ;
- utiliser des tags et politiques pour contrôler les ressources.

15.2 Exemples de risques

Risque	Impact	Mesure corrective
State exposé	Fuite d'informations sensibles.	Backend distant sécurisé, RBAC, chiffrement.
SSH ouvert à Internet	Risque d'attaque brute-force.	Filtrage par IP, Bastion, désactivation SSH public.
Secrets dans Git	Compromission d'identifiants.	Key Vault, variables sécurisées, scan de secrets.
Droits trop larges	Destruction ou modification non contrôlée.	Rôles limités au périmètre requis.
Absence de tags	Coûts difficiles à piloter.	Politique de tagging obligatoire.

16 FinOps et Terraform

Terraform contribue à la maîtrise des coûts parce qu'il rend les ressources visibles, nommées, taguées et destructibles. Toutefois, Terraform ne garantit pas à lui seul l'optimisation financière. Il faut compléter l'IaC par une gouvernance FinOps.

16.1 Leviers FinOps applicables au TP2

- choisir des tailles de VM adaptées à l'environnement ;
- utiliser des tags de coût ;
- détruire les environnements temporaires ;
- éviter les IP publiques inutiles ;

- documenter les ressources payantes ;
- comparer plusieurs options de stockage ;
- utiliser Azure Pricing Calculator avant le déploiement.

16.2 Exemple d'arbitrage

Pour un environnement de développement, une VM de petite taille peut suffire. Pour un environnement de production, la disponibilité, la performance et la redondance peuvent justifier un coût supérieur. L'objectif n'est pas de minimiser les coûts à tout prix, mais d'obtenir un rapport coût / valeur acceptable.

17 Terraform, CI/CD et travail en équipe

17.1 Workflow Git recommandé

Un workflow professionnel peut suivre les étapes suivantes :

1. création d'une branche ;
2. modification des fichiers Terraform ;
3. exécution de `terraform fmt` et `terraform validate` ;
4. génération d'un `terraform plan` ;
5. revue de code par un pair ;
6. validation sécurité et FinOps si nécessaire ;
7. application via pipeline contrôlé ;
8. archivage du plan et des logs.

17.2 Rôles dans l'équipe

Rôle	Responsabilité
Développeur	Propose des changements nécessaires à l'application.
DevOps / Cloud Engineer	Structure le code Terraform et maintient les modules.
Architecte	Valide la cohérence avec les principes d'architecture.
RSSI / Sécurité	Vérifie les droits, les ouvertures réseau et la protection des secrets.
FinOps	Suit les coûts, les tags et les dérives budgétaires.

18 Comparaison Terraform, ARM, Bicep et scripts

Outil	Forces	Limites	Cas d'usage
Terraform	Multi-cloud, large écosystème, modules, plan lisible.	Nécessite une gestion rigoureuse du state.	Organisations multi-cloud ou standard IaC transverse.
ARM Templates	Natif Azure, complet.	Syntaxe JSON souvent verbeuse.	Déploiements Azure très proches de l'API.

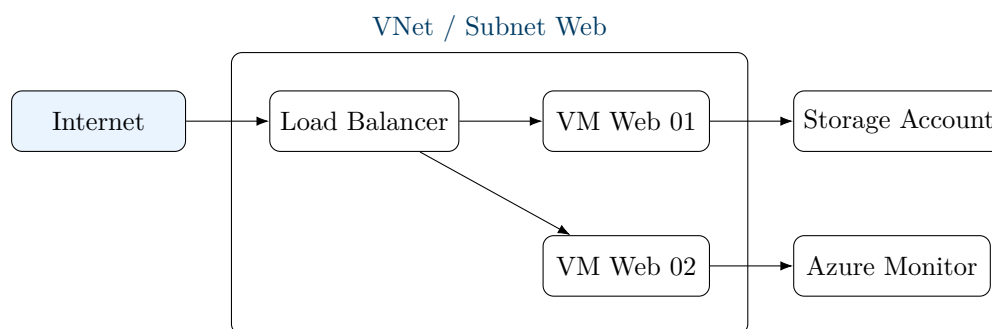
Bicep	Natif Azure, plus lisible qu'ARM.	Moins multi-cloud.	Équipes centrées Azure.
Scripts CLI	Simple pour actions ponctuelles.	Peu déclaratifs, moins reproductibles.	Administration, dépannage, automatisations ciblées.

À retenir

Terraform n'est pas le seul outil IaC. Son intérêt majeur dans ce module est de montrer une approche déclarative, multi-cloud et largement utilisée dans les équipes DevOps.

19 Architecture cible du TP2

Le TP2 vise à transformer une architecture logique en infrastructure Azure déployable par Terraform.



L'architecture pédagogique est volontairement limitée. L'objectif n'est pas de reproduire une production complète, mais de manipuler les concepts essentiels : réseau, sécurité, calcul, stockage, variables, outputs, state et contrôle des changements.

20 Méthode de lecture d'un plan Terraform

Lire un plan Terraform est une compétence importante. L'apprenant doit répondre aux questions suivantes :

- Quelles ressources vont être créées ?
- Quelles ressources vont être modifiées ?
- Une ressource va-t-elle être détruite ou remplacée ?
- Les noms et régions sont-ils conformes ?
- Les règles réseau sont-elles acceptables ?
- Les tags obligatoires sont-ils présents ?
- Les coûts potentiels sont-ils cohérents avec l'environnement ?

Question de vérification

Un plan affiche `-/+ azurerm_linux_virtual_machine.web`. Que signifie ce symbole ? Réponse : la VM sera remplacée, donc détruite puis recrée. Cela peut entraîner une interruption de service et une perte de données locales si elles ne sont pas externalisées.

21 Préparation directe au TP2

Avant de commencer le TP, l'apprenant doit vérifier :

- l'accès à une subscription Azure ou à un environnement de formation ;
- l'installation de Terraform ;
- l'installation d'Azure CLI ;
- la connexion avec `az login` ;
- la présence d'une clé SSH ;
- la compréhension des fichiers `providers.tf`, `variables.tf`, `main.tf` et `outputs.tf` ;
- la capacité à lire un plan avant application.

22 Mini-cas d'application

La société ShopEasy souhaite créer un environnement de développement pour tester une nouvelle version de son application. L'équipe doit pouvoir recréer cet environnement à la demande et le supprimer après usage.

22.0 Questions

1. Pourquoi Terraform est-il adapté à ce besoin ?
2. Quelles ressources Azure minimales faut-il créer ?
3. Quels tags sont indispensables ?
4. Quels risques apparaissent si le state reste local ?
5. Quelle règle NSG faut-il éviter ?
6. Quelles sorties Terraform seraient utiles aux développeurs ?

22.0 Éléments de réponse attendus

Terraform est adapté car il rend l'environnement reproductible et destructible. Les ressources minimales incluent Resource Group, VNet, subnet, NSG, VM, NIC, éventuellement Load Balancer et Storage Account. Les tags doivent inclure projet, environnement, propriétaire et centre de coût. Le state local est risqué en équipe. Il faut éviter SSH ouvert à Internet. Les outputs utiles incluent IP publique, nom du Resource Group et URL de test.

23 Questions de vérification

1. Quelle différence entre une infrastructure créée manuellement et une infrastructure décrite en IaC ?
2. Pourquoi Terraform est-il qualifié de déclaratif ?
3. Quel est le rôle du provider AzureRM ?
4. À quoi sert `terraform init` ?
5. Pourquoi faut-il lire `terraform plan` avant `terraform apply` ?
6. Quel est le rôle du fichier state ?
7. Pourquoi le state local est-il problématique en équipe ?
8. Quelle différence entre variable, local et output ?
9. Pourquoi utiliser des tags sur toutes les ressources ?
10. Qu'est-ce que le drift ?

11. Pourquoi éviter les secrets dans les fichiers Terraform ?
12. Quel est l'intérêt des modules ?
13. Pourquoi limiter l'ouverture SSH ?
14. Comment Terraform peut-il contribuer au FinOps ?
15. Quelle est la différence entre Terraform et un script Azure CLI ?

24 Glossaire

Terme	Définition
IaC	Infrastructure as Code : gestion d'infrastructure par fichiers versionnés.
Terraform	Outil IaC déclaratif développé par HashiCorp.
Provider	Plugin permettant à Terraform de piloter une plateforme.
AzureRM	Provider Terraform standard pour Azure Resource Manager.
HCL	Langage de configuration utilisé par Terraform.
State	Fichier décrivant les ressources connues et gérées par Terraform.
Backend	Emplacement de stockage du state.
Plan	Prévisualisation des changements Terraform.
Apply	Application des changements prévus.
Drift	Écart entre le code Terraform et l'infrastructure réelle.
Module	Ensemble réutilisable de configurations Terraform.
Resource Group	Conteneur logique de ressources Azure.
VNet	Réseau privé virtuel Azure.
NSG	Groupe de sécurité réseau filtrant le trafic.
RBAC	Contrôle d'accès basé sur les rôles.
FinOps	Pratiques de pilotage et d'optimisation des coûts cloud.

25 Synthèse pédagogique

Le TP2 marque le passage d'une logique d'architecture à une logique d'industrialisation. Terraform permet de rendre l'infrastructure Azure reproductible, traçable et contrôlable. Cette approche améliore la qualité d'exploitation du SI, mais exige de bonnes pratiques : gestion rigoureuse du state, sécurité des identités, lecture systématique du plan, convention de nommage, tags et contrôle du drift.

À retenir

Le résultat attendu n'est pas seulement une infrastructure qui fonctionne. Le résultat attendu est une infrastructure décrite proprement, compréhensible, versionnable, sécurisée et maîtrisée dans son cycle de vie.

26 Références officielles utiles

— Microsoft Learn – Terraform on Azure documentation : <https://learn.microsoft.com/en-us/azure/developer/terraform/>

- Microsoft Learn – Store Terraform state in Azure Storage : <https://learn.microsoft.com/en-us/azure/developer/terraform/store-state-in-azure-storage>
- Microsoft Learn – Azure Well-Architected Framework : <https://learn.microsoft.com/en-us/azure/well-architected/>
- HashiCorp Developer – Terraform language, input variables and outputs : <https://developer.hashicorp.com/terraform/language>
- HashiCorp Developer – Backend azurerm : <https://developer.hashicorp.com/terraform/language/backend/azurerm>